Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 4
«Дослідження та реалізація криптографічних механізмів»
з дисципліни «Безпека програмного забезпечення»

Варіант 12

Виконав: здобувач групи ІП-33
Ломакін Максим

Перевірив:
Курченко Олег Анастасійович

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи – набути практичних навичок застосування криптографічних примітивів для забезпечення конфіденційності, цілісності та автентичності даних і навчитися розпізнавати типові помилки їх використання у програмному коді.

Завдання: виконати покрокову верифікацію навчальної реалізації RSA на малих числах; реалізувати автентифіковане шифрування за схемою AEAD варіанта з унікальним nonce та прив'язкою асоційованих даних; показати наслідки двох типових помилок застосування криптографії; сформувати й перевірити електронний підпис; обґрунтувати висновки про криптостійкість отриманих рішень.

Варіант 12. Схема AEAD – ChaCha20-Poly1305; схема підпису – RSA-PSS (2048); розрядність множників демонстраційного RSA – 32 біти; помилки для демонстрації – «MAC-then-Encrypt замість Encrypt-then-MAC» і «статичний вектор ініціалізації в CBC». Вивід команди `python3 -m common.variants 12 --lab 4`:

```
Варіант 12

  LAB4
    aead: ChaCha20-Poly1305
    signature: RSA-PSS (2048)
    rsa_demo_bits: 32
    misuse_to_demonstrate:
      - MAC-then-Encrypt замість Encrypt-then-MAC
      - статичний вектор ініціалізації в CBC
```

Повторний запуск дає той самий набір параметрів. Залежності встановлено за `requirements.txt` (cryptography 43.0.1, pytest 8.3.3). Усі ключі й дані в роботі синтетичні.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Шифрування захищає від перехоплення, код автентифікації повідомлення (MAC) – від модифікації; це різні загрози, і шифр без контролю цілісності не зупиняє активного порушника, який змінює шифротекст у каналі [1]. Тому для симетричного захисту обрано автентифіковане шифрування з асоційованими даними (AEAD): один примітив дає і конфіденційність, і цілісність, а тег перевіряється до видачі відкритого тексту. Варіант задає ChaCha20-Poly1305 [4]: потоковий шифр ChaCha20 з 256-бітовим ключем і 96-бітовим nonce та одноразовий код Poly1305 із 128-бітовим тегом. Стійкість схеми тримається на

унікальності nonce за фіксованого ключа, тому в реалізації nonce щоразу береться з генератора ОС: для випадкових 96-бітових значень імовірність збігу серед q повідомлень становить близько $q^2/2^{97}$, тобто 2^{-33} навіть для 2^{32} повідомлень під одним ключем.

Режим CBC обчислює $c_1 = E(k, m_1 \oplus IV)$, $c_i = E(k, m_i \oplus c_{i-1})$. Блоковий шифр E детермінований, тож єдине джерело випадковості – вектор ініціалізації; стандарт вимагає, щоб IV був непередбачуваним і новим для кожного повідомлення [8]. За незмінного IV однакові перші блоки відкритого тексту дають однакові блоки шифротексту, і збіг триває до першого відмінного блоку – саме цей наслідок перевіряється в роботі.

Шифр і MAC можна поєднати у двох порядках. За MAC-then-Encrypt код обчислюють над відкритим текстом і шифрують разом із ним, тому приймач мусить спочатку розшифрувати неперевірені дані й розібрати їх доповнення, і лише потім може перевірити MAC. За Encrypt-then-MAC код обчислюють над шифротекстом, і приймач відкидає змінені дані до розшифрування. Доведено, що з трьох типових композицій лише Encrypt-then-MAC гарантує і конфіденційність, і цілісність шифротексту для будь-яких стійких складників [7]; на різниці відповідей приймача MAC-then-Encrypt побудовано клас атак на доповнення [10]. Це обґрунтовує вибір виправленої конструкції: MAC над IV і шифротекстом, окремий ключ для MAC, порівняння тегів за сталий час.

RSA [3] спирається на складність факторизації $n = p \cdot q$: закриту експоненту $d \equiv e^{-1} \pmod{\varphi(n)}$ легко обчислити розширеним алгоритмом Евкліда, якщо відомі p і q. Тому розрядність множників визначає стійкість, і навчальні 32-бітові множники варіанта придатні лише для перевірки алгоритму. Тест Міллера–Рабіна імовірнісний: складене число проходить один раунд з імовірністю не більше 1/4, тож 20 раундів каркаса дають похибку не більше $4^{-20} \approx 10^{-12}$ [2]. Піднесення до степеня методом «квадрат і множення» потребує не більше 2k множень для k-бітового показника.

Для підпису варіант задає RSA-PSS з модулем 2048 бітів [5]. Підпис обчислюється над хеш-сумою SHA-256, а доповнення PSS додає випадкову сіль, тому підпис імовірнісний і має доведення стійкості, на відміну від «підручникового» RSA без доповнення. Модуль 2048 бітів відповідає рівню стійкості 112 бітів і є мінімальним допустимим [6]. На відміну від MAC, підпис перевіряється відкритим ключем і не потребує спільного секрету.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура рішення

Рішення складається з трьох груп модулів пакета `crypto_lab` (рисунок 3.1). Модуль `rsa_edu.py` узято з репозиторію курсу [11] без змін; у каркасі `aead.py` реалізовано функції `encrypt` і `decrypt`; решту модулів написано в межах роботи (таблиця 3.1). Криптографічні примітиви надає бібліотека cryptography: власноруч реалізовано лише їх поєднання.

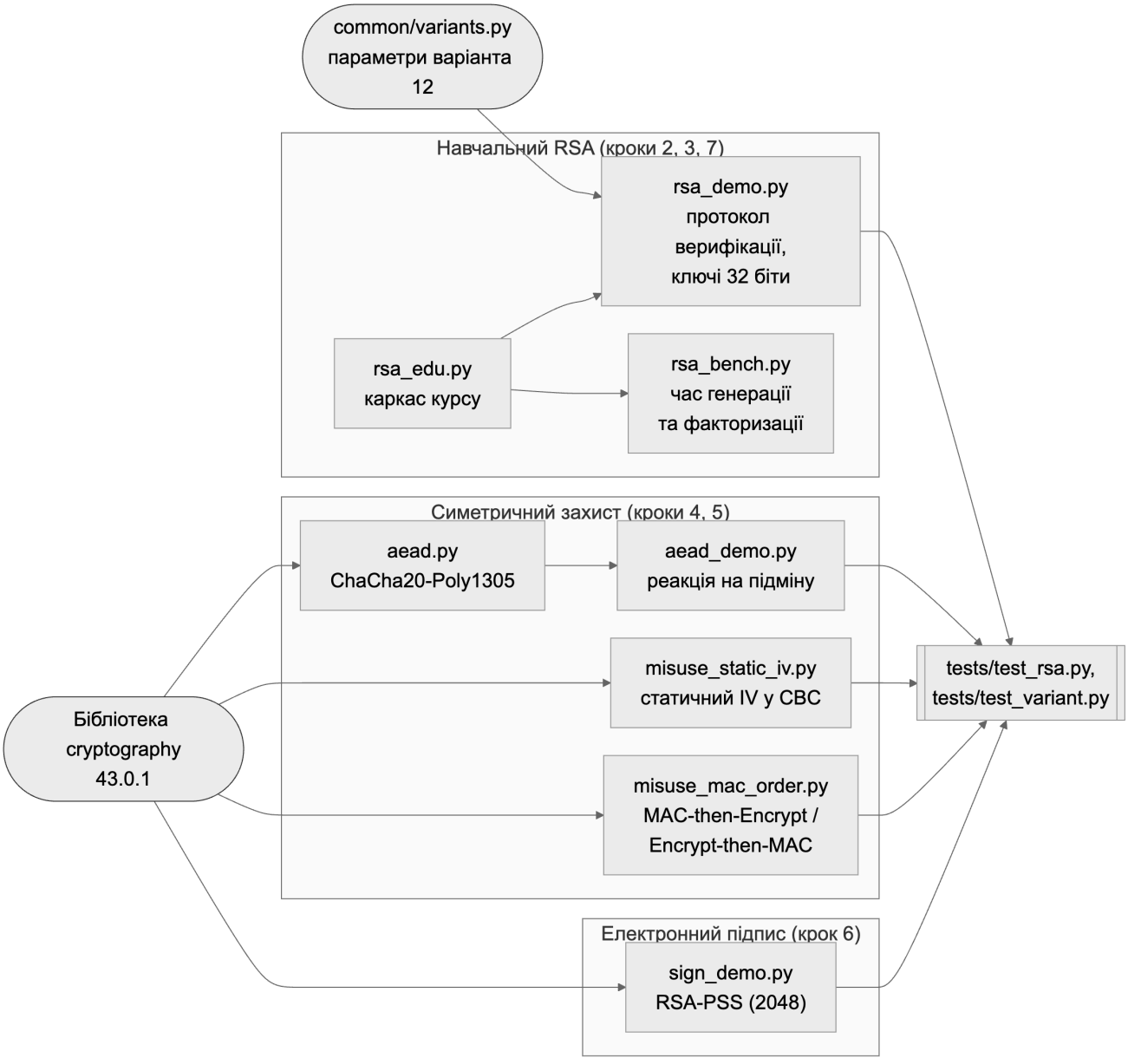


Рисунок 3.1 – Структурна схема рішення

Таблиця 3.1 – Модулі рішення

Файл	Крок	Вміст
`rsa_edu.py`	2, 3	каркас курсу: `fast_pow_mod`, `is_probable_prime`, `egcd`, `modinv`, `keygen`
`rsa_demo.py`	2, 3	протокол верифікації RSA, ключі з 32-бітовими множниками
`aead.py`	4	`encrypt` і `decrypt` за схемою ChaCha20-Poly1305
`aead_demo.py`	4	оборотність, унікальність nonce, реакція на підміну
`misuse_static_iv.py`	5	CBC зі статичним і з випадковим IV
`misuse_mac_order.py`	5	MAC-then-Encrypt і Encrypt-then-MAC
`sign_demo.py`	6	підпис RSA-PSS (2048) і його перевірка
`rsa_bench.py`	7	час генерації простих чисел і факторизації модуля
`tests/test_variant.py`	2–6	32 власні тести

3.2 Автентифіковане шифрування

```
def encrypt(plaintext: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """БЕЗПЕЧНО: автентифіковане шифрування ChaCha20-Poly1305.

    Вимоги:
      - унікальний nonce на кожне повідомлення (лекція 10);
      - прив'язка асоційованих даних;
      - повернення nonce разом із шифротекстом і тегом.
    """
    nonce = os.urandom(NONCE_SIZE)
    sealed = ChaCha20Poly1305(key).encrypt(nonce, plaintext, associated)
    return nonce + sealed


def decrypt(blob: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """Перевіряє тег і повертає відкритий текст; за розбіжності — InvalidTag."""
    if len(blob) < NONCE_SIZE + TAG_SIZE:
        raise ValueError("Повідомлення коротше за nonce і тег")
    nonce, sealed = blob[:NONCE_SIZE], blob[NONCE_SIZE:]
    return ChaCha20Poly1305(key).decrypt(nonce, sealed, associated)
```

Унікальність nonce. Значення nonce не приймається від викликача й не зберігається між викликами: кожен виклик `encrypt` бере 12 нових байтів з генератора ОС (`os.urandom`). Тому повторити nonce помилкою у викликаючому коді неможливо, а лічильник, який довелося б зберігати між запусками, не потрібен. Nonce не є секретом і передається першими 12 байтами пакета.

Прив'язка асоційованих даних. Параметр `associated` передається в `ChaCha20Poly1305.encrypt` третім аргументом: дані не шифруються й до пакета не входять, але тег Poly1305 обчислюється над ними та над шифротекстом. Під час розшифрування приймач подає ті самі дані; якщо вони відрізняються хоча б одним бітом, тег не збігається. Функція `decrypt` не перехоплює виняток `InvalidTag`,

тому за розбіжності викликач не отримує жодних даних. Пакет, коротший за 28 байтів, відхиляється ще до звернення до бібліотеки.

3.3 Статичний вектор ініціалізації в CBC

Обидва варіанти використовують AES-256-CBC з доповненням PKCS7 і відрізняються лише джерелом IV:

```
def encrypt_static_iv(plaintext: bytes, key: bytes) -> bytes:
    """Шифрування AES-CBC з незмінним вектором ініціалізації."""
    # ВРАЗЛИВО: той самий IV для всіх повідомлень – шифрування детерміноване
    return _cbc_encrypt(plaintext, key, STATIC_IV)


def encrypt_random_iv(plaintext: bytes, key: bytes) -> bytes:
    """Шифрування AES-CBC; результат: IV ‖ шифротекст."""
    # БЕЗПЕЧНО: новий випадковий IV на кожне повідомлення, IV передається відкрито
    iv = os.urandom(BLOCK)
    return iv + _cbc_encrypt(plaintext, key, iv)
```

У вразливому варіанті IV – константа з 16 нульових байтів, тож результат залежить лише від ключа й повідомлення. У виправленому IV генерується для кожного повідомлення й передається відкрито перед шифротекстом. Виправлений варіант усуває саме цю помилку, але цілісності не дає; для нового коду слід брати AEAD з підрозділу 3.2.

3.4 Порядок шифрування та обчислення MAC

Обидві схеми побудовано з тих самих складників: AES-256-CBC, HMAC-SHA-256, випадковий IV. Відрізняється лише порядок дій. Кожен приймач записує виконані етапи в перелік `trace`, щоб порядок було видно у виводі. Вразлива схема:

```
def mte_seal(plaintext: bytes, enc_key: bytes, mac_key: bytes) -> bytes:
    """MAC-then-Encrypt: IV ‖ CBC(відкритий текст ‖ MAC)."""
    # ВРАЗЛИВО: MAC обчислено над відкритим текстом і сховано під шифруванням
    iv = os.urandom(BLOCK)
    return iv + _cbc_encrypt(plaintext + _mac(mac_key, plaintext), enc_key, iv)


def mte_open(blob: bytes, enc_key: bytes, mac_key: bytes,
             trace: list[str]) -> bytes:
    # ВРАЗЛИВО: приймач розшифровує неперевірені дані, MAC перевіряється останнім
    iv, body = blob[:BLOCK], blob[BLOCK:]
    data = _cbc_decrypt(body, enc_key, iv, trace)
    trace.append(CHECK)
    if len(data) < TAG:
        raise MacError("код автентифікації не збігається")
    plaintext, tag = data[:-TAG], data[-TAG:]
    if not hmac.compare_digest(tag, _mac(mac_key, plaintext)):
        raise MacError("код автентифікації не збігається")
    return plaintext
```

Функція `_cbc_decrypt` розшифровує дані, знімає доповнення і за некоректного доповнення підіймає `PaddingError`. Отже, для зміненого пакета приймач спершу виконує розшифрування й розбір доповнення, а до перевірки MAC доходить не завжди. Виправлена схема:

```
def etm_seal(plaintext: bytes, enc_key: bytes, mac_key: bytes) -> bytes:
    """Encrypt-then-MAC: IV ‖ CBC(відкритий текст) ‖ MAC(IV ‖ шифротекст)."""
    # БЕЗПЕЧНО: MAC покриває IV і шифротекст, ключ MAC відмінний від ключа шифру
    iv = os.urandom(BLOCK)
    body = iv + _cbc_encrypt(plaintext, enc_key, iv)
    return body + _mac(mac_key, body)


def etm_open(blob: bytes, enc_key: bytes, mac_key: bytes,
             trace: list[str]) -> bytes:
    # БЕЗПЕЧНО: MAC перевіряється першим і за сталий час; змінені дані
    # відхиляються до розшифрування
    trace.append(CHECK)
    if len(blob) < 2 * BLOCK + TAG:
        raise MacError("код автентифікації не збігається")
    body, tag = blob[:-TAG], blob[-TAG:]
    if not hmac.compare_digest(tag, _mac(mac_key, body)):
        raise MacError("код автентифікації не збігається")
    return _cbc_decrypt(body[BLOCK:], enc_key, body[:BLOCK], trace)
```

MAC покриває IV і шифротекст, обчислюється окремим ключем і порівнюється функцією `hmac.compare_digest` за сталий час. Розшифрування викликається лише в останньому рядку `etm_open`, тобто тільки для автентичних пакетів.

3.5 Електронний підпис

```
PSS = padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                  salt_length=padding.PSS.MAX_LENGTH)


def generate_keys() -> tuple[rsa.RSAPrivateKey, rsa.RSAPublicKey]:
    # ТЕСТОВИЙ КЛЮЧ — не використовувати поза стендом
    private = rsa.generate_private_key(public_exponent=65537, key_size=KEY_BITS)
    return private, private.public_key()


def sign(private: rsa.RSAPrivateKey, message: bytes) -> bytes:
    return private.sign(message, PSS, hashes.SHA256())


def verify(public: rsa.RSAPublicKey, signature: bytes, message: bytes) -> bool:
    """True, якщо підпис відповідає повідомленню й відкритому ключу."""
    try:
        public.verify(signature, message, PSS, hashes.SHA256())
        return True
    except InvalidSignature:
        return False
```

Ключ генерує бібліотека: модуль 2048 бітів, e = 65537. Підпис обчислюється над хеш-сумою SHA-256 з доповненням PSS, функцією генерування маски MGF1 і

сіллю найбільшої довжини. Функція `verify` перетворює виняток `InvalidSignature` на значення `False`, щоб результат перевірки можна було порахувати для серії змінених повідомлень.

3.6 Засоби верифікації RSA

Модуль `rsa_demo.py` не дублює каркас, а викликає його функції й друкує проміжні значення. Додано три допоміжні функції: `egcd_trace` – ітеративний запис розширеного алгоритму Евкліда з рядками (q, r, x, y), для кожного з яких перевіряється рівність $a \cdot x + b \cdot y = r$; `miller_rabin_trace` – розклад $n - 1 = 2^r \cdot d$ і послідовність a^d, a^(2d), … для заданої основи a; `multiplications` – підрахунок піднесень до квадрата й добутків у протоколі `fast_pow_mod`.

```
def egcd_trace(a: int, b: int) -> list[tuple[int, int, int, int]]:
    """Рядки (q, r, x, y) розширеного алгоритму Евкліда: a·x + b·y = r."""
    rows = [(0, a, 1, 0), (0, b, 0, 1)]
    while rows[-1][1]:
        (_, r0, x0, y0), (_, r1, x1, y1) = rows[-2], rows[-1]
        q = r0 // r1
        rows.append((q, r0 - q * r1, x0 - q * x1, y0 - q * y1))
    return rows[:-1]
```

Модуль `rsa_bench.py` вимірює час `gen_prime` і час розкладання власного модуля двома методами: перебором непарних дільників і ρ-методом Полларда. Обидва застосовуються лише до модулів, згенерованих у цій роботі.

4 РЕЗУЛЬТАТИ

4.1 Покрокова верифікація RSA

Протокол `fast_pow_mod` для $7^{13} \bmod 100$: показник $13 = 1101_2$ обробляється від молодшого біта, у стовпці `b` – послідовні квадрати основи, у `result` – накопичений добуток.

```
7^13 mod 100 = 7
  e=1101 bit=1 b=7 result=7
  e= 110 bit=0 b=49 result=7
  e=  11 bit=1 b=1 result=7
  e=   1 bit=1 b=1 result=7
  бітів показника: 4, з них одиничних: 3; множень: 7 (4 квадр. + 3 добутків)
```

Результат 7 збігається з очікуваним ($7^4 \equiv 1 \bmod 100$). Тест простоти:

```
is_probable_prime(7919) = True
is_probable_prime(7917) = False
is_probable_prime(104729) = True
is_probable_prime(2047) = False
is_probable_prime(1373653) = False
is_probable_prime(3215031751) = False
n=104729: n-1 = 2^3·13091, a=2, x = [36639, 104728, 1] -> можливо просте
n=2047: n-1 = 2^1·1023, a=2, x = [1] -> можливо просте
n=2047: n-1 = 2^1·1023, a=3, x = [1565] -> складене
n=1373653: n-1 = 2^2·343413, a=2, x = [890592, 1373652] -> можливо просте
n=1373653: n-1 = 2^2·343413, a=5, x = [1199564, 73782] -> складене
```

Числа 7919 і 104729 прості, 7917 = 3·7·13·29 відкидається ще перевіркою малих дільників. Останні п'ять рядків показують окремі раунди: число проходить раунд, якщо перше значення x дорівнює 1 або серед значень є n − 1. Складене 2047 = 23·89 проходить раунд з основою 2, але не з основою 3; складене 1373653 – з основою 2, але не з основою 5. Тому тест повторюють з випадковими основами: `is_probable_prime` з 20 раундами відкинув усі три такі числа.

Розширений алгоритм Евкліда та розрахунок закритої експоненти:

```
egcd(240, 46) = (2, -9, 47); 240·(-9) + 46·(47) = 2
обчислення d для e = 17, φ = 3120 (3120·x + 17·y = r):
    q     r     x      y
    0  3120     1      0
    0    17     0      1
  183     9     1   -183
    1     8    -1    184
    1     1     2   -367
y = -367, d = y mod 3120 = 2753; modinv(17, 3120) = 2753
```

Останній рядок таблиці дає 3120·2 + 17·(−367) = 1, звідки 17·(−367) ≡ 1 (mod 3120) і d = −367 mod 3120 = 2753. Контрольний приклад p = 61, q = 53, e = 17:

```
p = 61, q = 53, n = 3233, φ(n) = 3120, e = 17, d = 2753
e·d mod φ(n) = 1
65^17 mod 3233 = 2790
  e=10001 bit=1 b=65 result=65
  e=1000 bit=0 b=992 result=65
  e= 100 bit=0 b=1232 result=65
  e=  10 bit=0 b=1547 result=65
  e=   1 bit=1 b=789 result=2790
  бітів показника: 5, з них одиничних: 2; множень: 7 (5 квадр. + 2 добутків)
2790^2753 mod 3233 = 65
  e=101011000001 bit=1 b=2790 result=2790
  e=10101100000 bit=0 b=2269 result=2790
  e=1010110000 bit=0 b=1425 result=2790
  e=101011000 bit=0 b=301 result=2790
  e=10101100 bit=0 b=77 result=2790
  e=1010110 bit=0 b=2696 result=2790
  e=101011 bit=1 b=632 result=1295
  e=10101 bit=1 b=1765 result=3177
  e=1010 bit=0 b=1846 result=3177
  e= 101 bit=1 b=134 result=2195
  e=  10 bit=0 b=1791 result=2195
  e=   1 bit=1 b=545 result=65
  бітів показника: 12, з них одиничних: 5; множень: 17 (12 квадр. + 5 добутків)
```

Усі значення збігаються з контрольними: n = 3233, φ(n) = 3120, d = 2753, c = 2790, розшифрування повертає 65. Еталонні тести каркаса: «6 passed in 0.01s».

4.2 Демонстраційні ключі RSA

Ключі згенеровано викликом `keygen(bits=32)`; значення наведено в таблиці 4.1.

Таблиця 4.1 – Демонстраційні ключі RSA з 32-бітовими множниками

Параметр	Значення	Бітів
p	4 104 977 939	32
q	4 229 742 271	32
$n = p \cdot q$	17 362 998 710 110 759 469	64
$\varphi(n) = (p - 1)(q - 1)$	17 362 998 701 776 039 260	64
e	65 537	17
$d = e^{-1} \bmod \varphi(n)$	10 218 515 646 543 202 073	64

```
розрядність у бітах: p - 32, q - 32, n - 64, d - 64
is_probable_prime(p) = True, is_probable_prime(q) = True
e·d mod φ(n) = 1
m = 42 -> c = 7321213873260625553 -> m = 42
```

Обидва множники мають по 32 біти й проходять тест простоти, e·d mod φ(n) = 1, шифрування m = 42 оборотне. Протокол піднесення до степеня e = 65537 і підсумок для показника d:

```
42^65537 mod 17362998710110759469 = 7321213873260625553
  e=10000000000000001 bit=1 b=42 result=42
  e=1000000000000000 bit=0 b=1764 result=42
  e=100000000000000 bit=0 b=3111696 result=42
  e=10000000000000 bit=0 b=9682651996416 result=42
  e=1000000000000 bit=0 b=15684620810546767524 result=42
  e=100000000000 bit=0 b=7930893707279100091 result=42
  e=10000000000 bit=0 b=6903345712341791013 result=42
  e=1000000000 bit=0 b=4378436061253914946 result=42
  e=100000000 bit=0 b=8788503072219235745 result=42
  e=10000000 bit=0 b=8893146552498034527 result=42
  e=1000000 bit=0 b=2077657257431231280 result=42
  e=100000 bit=0 b=6457012067945484099 result=42
  e=10000 bit=0 b=10044419289523919637 result=42
  e=1000 bit=0 b=4643490310674627838 result=42
  e= 100 bit=0 b=7958375899209881755 result=42
  e=  10 bit=0 b=3288825898034235508 result=42
  e=   1 bit=1 b=9682623433471621270 result=7321213873260625553
  бітів показника: 17, з них одиничних: 2; множень: 19 (17 квадр. + 2 добутків)
7321213873260625553^10218515646543202073 mod 17362998710110759469 = 42
  бітів показника: 64, з них одиничних: 38; множень: 102 (64 квадр. + 38 добутків)
```

4.3 Автентифіковане шифрування

Вхідні дані: повідомлення b'secret payload', асоційовані дані b'hdr', тестовий ключ із 32 байтів.

```
відкритий текст 14 байтів -> пакет 42 байтів: nonce 12 + шифротекст 14 + тег 16
nonce:      9e117a4b641c8bacba2b116b
шифротекст: a8909326fdb03fd8de57b77eee4f
тег:        2635b9951502f89cb7beef982f4a74cb
розшифрування: b'secret payload'
повторне шифрування дає інший nonce: True, інший шифротекст: True
різних nonce у 10000 шифруваннях: 10000
змінено біт тегу: InvalidTag, даних не видано
змінено біт шифротексту: InvalidTag, даних не видано
змінено біт nonce: InvalidTag, даних не видано
змінено біт асоційованих даних: InvalidTag, даних не видано
асоційовані дані відсутні: InvalidTag, даних не видано
інший ключ: InvalidTag, даних не видано
однобітових змін пакета відхилено: 336 із 336
-- лише шифрування, без тегу --
змінено біт шифротексту: приймач без помилки отримав b'recret payload'
```

Пакет на 28 байтів довший за відкритий текст (nonce і тег). Повторне шифрування того самого повідомлення дає інший пакет, у 10 000 шифрувань отримано 10 000 різних nonce. Усі 336 однобітових змін пакета відхилено. Останній рядок – контрольний дослід із шифруванням без тегу: змінений шифротекст прийнято, і приймач отримав b'recret payload'.

4.4 Наслідок статичного IV

Вхідні дані – три синтетичні повідомлення по 42–45 байтів (три блоки після доповнення): `draft` і `final` збігаються в перших 40 байтах і різняться останнім словом, `memo` відрізняється з першого блока. Для кожного блока шифротексту показано перші 8 байтів.

```
draft         ef003cb7990ddcbe… 1ca1d36d85a20f15… 3d1c77273f63d89c…
draft ще раз  ef003cb7990ddcbe… 1ca1d36d85a20f15… 3d1c77273f63d89c…
final         ef003cb7990ddcbe… 1ca1d36d85a20f15… 95a1fca8fd151284…
memo          61b6e349a7e536ce… ce8beeeacb40b656… 7469ffbffc89ba56…
однакові повідомлення: спільних перших блоків 3 із 3, шифротексти рівні: True
спільні перші 40 байтів: спільних перших блоків 2 із 3, шифротексти рівні: False
різні початки: спільних перших блоків 0 із 3, шифротексти рівні: False
серія з 5 повідомлень (3 різних): різних шифротекстів 3

draft         feaa0e16e7c14d9e… d034a7634201c1c1… d6965757118e9c3b…
draft ще раз  a991453bf85f2918… 9a3dd008f149d2df… dcbd3ec160566ffd…
final         5bbd8073322503ab… 3ff1ba070607d513… a3f27d5b90635f44…
memo          5632f4187b92b6e8… 4583bcc79028bac1… 7b372d5d6b9cf557…
однакові повідомлення: спільних перших блоків 0 із 3, шифротексти рівні: False
спільні перші 40 байтів: спільних перших блоків 0 із 3, шифротексти рівні: False
різні початки: спільних перших блоків 0 із 3, шифротексти рівні: False
серія з 5 повідомлень (3 різних): різних шифротекстів 5
розшифрування відновлює всі повідомлення серії: True
```

За статичного IV повторне шифрування `draft` дає той самий шифротекст, а `draft` і `final` мають 2 однакові перші блоки; третій блок різний, бо відмінність починається з 41-го байта. У серії з 5 повідомлень лише 3 різні шифротексти. За

випадкового IV жодного спільного блока немає, усі 5 шифротекстів різні, розшифрування відновлює всі повідомлення.

4.5 Наслідок порядку MAC і шифрування

Вхідні дані – синтетичне повідомлення з 40 байтів; у кожній схемі пакет має 96 байтів. Приймачеві подано незмінений пакет, два пакети з одним зміненим бітом, а далі – серію, у якій по черзі змінено молодший біт кожного байта пакета.

```
повідомлення 40 байтів, пакет 96 байтів
без змін: прийнято
  етапи приймача: розшифрування -> зняття доповнення -> перевірка MAC
змінено біт у першому блоці шифротексту: відмова: MAC
  етапи приймача: розшифрування -> зняття доповнення -> перевірка MAC
змінено біт у передостанньому блоці: відмова: доповнення
  етапи приймача: розшифрування -> зняття доповнення
зміна біта в кожному з 96 байтів: відмова: MAC – 72, відмова: доповнення – 24
розшифрувань неавтентичних даних: 96 із 96
```

```
повідомлення 40 байтів, пакет 96 байтів
без змін: прийнято
  етапи приймача: перевірка MAC -> розшифрування -> зняття доповнення
змінено біт у першому блоці шифротексту: відмова: MAC
  етапи приймача: перевірка MAC
змінено біт у передостанньому блоці: відмова: MAC
  етапи приймача: перевірка MAC
зміна біта в кожному з 96 байтів: відмова: MAC – 96
розшифрувань неавтентичних даних: 0 із 96
```

У схемі MAC-then-Encrypt приймач розшифрував усі 96 змінених пакетів і дав два різні результати: відмов через доповнення – 24 (до перевірки MAC справа не дійшла), відмов через MAC – 72. У схемі Encrypt-then-MAC усі 96 пакетів відхилено на першому ж етапі однією відмовою, розшифрувань – 0. Незмінені пакети обидві схеми приймають.

4.6 Електронний підпис

```
модуль: 2048 бітів, e = 65537
повідомлення: b'lab4 demo message'
підпис: 256 байтів, 5dcbea1c9811d36750e296b9f83fb66f6a7245e745b75c39…
коректне повідомлення: підпис дійсний
змінене повідомлення (msg + b'x'): InvalidSignature
змінено один біт повідомлення: InvalidSignature
змінено один біт підпису: InvalidSignature
однобітових змін підпису відхилено: 2048 із 2048
однобітових змін повідомлення відхилено: 136 із 136
перевірка чужим відкритим ключем: InvalidSignature
повторний підпис того самого повідомлення відрізняється: True, дійсний: True
```

Перевірка коректного підпису завершується без винятку. Зміна повідомлення (дописаний байт або один біт) і зміна одного біта підпису дають `InvalidSignature`; відхилено всі 2048 однобітових змін підпису й усі 136 змін

повідомлення. Підпис має 256 байтів – стільки ж, скільки модуль. Два підписи того самого повідомлення різні, і обидва дійсні: це наслідок випадкової солі PSS.

4.7 Час генерації простих чисел і факторизації

Таблиця 4.2 містить середній час `gen_prime`, таблиця 4.3 – час розкладання модуля $n = p \cdot q$, згенерованого `keygen`, перебором непарних дільників і ρ-методом Полларда. Виміри виконано на одному ядрі інтерпретатора Python 3.12; значення залежать від випадкових p і q.

Таблиця 4.2 – Середній час генерації простого числа

Розрядність, біт	Кількість спроб	Час, мс
16	200	0,047
32	200	0,129
64	200	0,324
128	20	1,243
256	20	7,319
512	5	40,096
1024	5	413,889

Таблиця 4.3 – Час факторизації навчального модуля

Біт множника	Біт модуля	Ділень (перебір)	Перебір, с	Ітерацій ρ	ρ-метод, с
16	32	22 853	0,0012	160	0,0001
20	40	364 706	0,0195	453	0,0002
24	48	6 355 326	0,3424	2 068	0,0011
28	56	90 586 524	4,8921	15 605	0,0083
32	64	1 828 623 411	159,0763	37 057	0,0263
36	71	–	–	189 880	0,1350
40	80	–	–	272 970	0,2050

Для множників, більших за 32 біти, перебір не запускався: за оцінкою він тривав би від 10,0 год для 40-бітових множників.

4.8 Модульні тести

Файл `tests/test_variant.py` містить 32 тести: AEAD (оборотність, формат пакета, унікальність nonce, усі однобітові зміни, асоційовані дані, ключ, обрізаний пакет), статичний і випадковий IV, обидва порядки MAC, підпис і допоміжні функції RSA. Запуск: `PYTHONPATH=. pytest tests/test_variant.py -v`.

```
============================= test session starts ==============================
collecting ... collected 32 items

tests/test_variant.py::test_aead_roundtrip_with_associated_data PASSED
tests/test_variant.py::test_aead_blob_layout PASSED
tests/test_variant.py::test_aead_empty_message PASSED
tests/test_variant.py::test_aead_nonce_is_unique_per_message PASSED
tests/test_variant.py::test_aead_rejects_any_single_bit_change PASSED
tests/test_variant.py::test_aead_binds_associated_data PASSED
tests/test_variant.py::test_aead_rejects_wrong_key PASSED
tests/test_variant.py::test_aead_rejects_truncated_blob PASSED
tests/test_variant.py::test_aead_rejects_wrong_key_length PASSED
tests/test_variant.py::test_static_iv_is_deterministic PASSED
tests/test_variant.py::test_static_iv_reveals_common_prefix PASSED
tests/test_variant.py::test_static_iv_reveals_repeats_in_series PASSED
tests/test_variant.py::test_random_iv_hides_repeats_and_prefixes PASSED
tests/test_variant.py::test_random_iv_roundtrip PASSED
tests/test_variant.py::test_mac_schemes_roundtrip[mte_seal-mte_open] PASSED
tests/test_variant.py::test_mac_schemes_roundtrip[etm_seal-etm_open] PASSED
tests/test_variant.py::test_mte_decrypts_before_authentication PASSED
tests/test_variant.py::test_mte_has_two_distinguishable_failures PASSED
tests/test_variant.py::test_etm_rejects_before_decryption PASSED
tests/test_variant.py::test_etm_mac_covers_iv PASSED
tests/test_variant.py::test_etm_rejects_truncated_blob PASSED
tests/test_variant.py::test_etm_requires_matching_mac_key PASSED
tests/test_variant.py::test_signature_valid PASSED
tests/test_variant.py::test_signature_rejects_changed_message PASSED
tests/test_variant.py::test_signature_rejects_changed_signature PASSED
tests/test_variant.py::test_signature_rejects_other_key PASSED
tests/test_variant.py::test_pss_signature_is_randomized PASSED
tests/test_variant.py::test_demo_keys_match_variant_bits PASSED
tests/test_variant.py::test_multiplication_count PASSED
tests/test_variant.py::test_egcd_trace_gives_private_exponent PASSED
tests/test_variant.py::test_miller_rabin_trace PASSED
tests/test_variant.py::test_factorization_of_small_modulus PASSED

============================== 32 passed in 0.41s ==============================
```

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

1. Час генерації простих чисел і факторизації. Генерація простого числа каркасом дешева і зростає поліноміально: 0,047 мс для 16 бітів, 0,129 мс для 32, 7,32 мс для 256 і 414 мс для 1024 бітів (таблиця 4.2) – розрядність зросла в 64 рази, час приблизно в 8 800 разів. Факторизація зростає експоненційно. Перебір дільників потребує близько p/2 ділень: 22 853 ділень і 1,2 мс для 16-бітових множників, 6 355 326 і 342,4 мс для 24-бітових, 1 828 623 411 і 2,7 хв для 32-бітових (таблиця 4.3); кожні 4 біти множника збільшують час приблизно в 16 разів. За виміряної швидкості 11,5 млн ділень за секунду оцінка для 40-бітових множників – 10,0 год, для 48-бітових – 106 діб, для 64-бітових – 19 082 роки. ρ-метод Полларда потребує близько $\sqrt{p}$ ітерацій, тому модуль варіанта (64 біти) розкладено за 26,3 мс (37 057 ітерацій), а 80-бітовий – за 205,0 мс. Вручну навчальний RSA непрактично факторизувати вже за 16-бітових множників:

потрібно близько 22 853 ділень 32-бітового числа, тоді як контрольний модуль 3233 розкладається за 26 ділень. Для комп'ютера всі розрядності роботи (16–40 бітів) нестійкі. Модуль 2048 бітів має множники по 1024 біти: перебір потребував би близько 2^{1023} ділень, ρ-метод – близько 2^{512} ітерацій, а найкращий відомий алгоритм (решето числового поля) дає трудомісткість близько 2^{112} операцій [6]. Рекордна факторизація 795-бітового модуля потребувала близько 900 процесоро-років [9], тож 2048 бітів лежать далеко за межею досяжного.

2. Кількість множень у fast_pow_mod. Для відкритого показника e = 65537 (17 бітів, 2 одиничні) функція виконала 19 множень за модулем: 17 піднесень до квадрата і 2 добутки. Для закритого показника d (64 біти, з них одиничних – 38) множень 102: квадратів – 64, добутків – 38. Загалом для k-бітового показника з w одиничними бітами реалізація каркаса робить k + w множень, тобто від k + 1 до 2k; останній квадрат і перший добуток на одиницю зайві, тож мінімально достатньо k + w − 2. Кількість множень лінійна від довжини показника в бітах, а не від його значення. Наївне піднесення потребує e − 1 = 65 536 множень для e і d − 1 ≈ $1{,}0 \cdot 10^{19}$ для d – за 10^9 множень за секунду це близько 324 роки замість 102 множень. Для 2048-бітового показника наївний метод потребує близько 2^{2048} множень, а «квадрат і множення» – не більше 4096. Крім того, без зведення за модулем на кожному кроці результат c^d мав би близько $1{,}9 \cdot 10^{20}$ десяткових розрядів.

3. Що показують демонстрації двох помилок. Статичний IV у CBC порушує конфіденційність за пасивного порушника, який лише спостерігає канал. Не знаючи ключа, він бачить, що два шифротексти збігаються повністю (3 блоки з 3) – отже, передано те саме повідомлення; що два шифротексти мають 2 спільні перші блоки – отже, повідомлення збігаються в перших 32 байтах; у серії з 5 повідомлень він нараховує 3 різні шифротексти й дізнається, які саме повідомлення повторювалися. За випадкового IV спільних блоків 0, а різних шифротекстів 5 із 5. Цілісності CBC не дає в обох випадках. **MAC-then-Encrypt** послаблює захист від активного порушника, який надсилає приймачеві змінені пакети. Підробку приймач відхиляє в обох схемах, тож цілісність формально збережено, але в MAC-then-Encrypt він розшифрував усі 96 із 96 змінених пакетів до перевірки автентичності й дав дві різні відповіді: «доповнення» – 24, «MAC» – 72. Яка з відповідей настане, залежить від розшифрованих секретних даних, тому відповідь приймача стає каналом витоку відомостей про відкритий текст – під

загрозою конфіденційність [10]. В Encrypt-then-MAC усі 96 змінених пакетів отримали одну відповідь, а розшифрувань було 0: відповідь залежить лише від шифротексту, який порушник і так знає.

4. AEAD проти шифрування без контролю цілісності. Шифрування без тегу приховує зміст, але приймає будь-який шифротекст. На кроці 4 це видно в останньому рядку виводу: після зміни одного біта шифротексту ChaCha20 без Poly1305 приймач без жодної помилки отримав b'recret payload' замість b'secret payload' – порушник змінив зміст, не знаючи ключа. Те саме видно на кроці 5: режим CBC без MAC не має засобу виявити зміну. AEAD додає 16-байтовий тег над nonce, асоційованими даними й шифротекстом: 336 із 336 однобітових змін пакета (nonce, шифротексту, тегу), а також зміна чи вилучення асоційованих даних і чужий ключ завершилися винятком `InvalidTag`, і жодного байта відкритого тексту не видано. Асоційовані дані при цьому передаються відкрито, але підмінити їх непомітно не можна.

5. Реакція на підміну одного біта; MAC і підпис. Обидві перевірки відхиляють будь-яку однобітову зміну. Тег AEAD відхилив 336 із 336 змін пакета; підпис RSA-PSS – 2048 із 2048 змін підпису та 136 із 136 змін повідомлення, а також дописаний байт і перевірку чужим відкритим ключем. Реакція однакова – виняток (`InvalidTag` або `InvalidSignature`) без часткового результату. Відмінність у гарантіях. Тег обчислюють і перевіряють тим самим секретним ключем, тому він підтверджує лише, що повідомлення створив хтось із власників ключа: приймач сам здатен виготовити такий тег, і третій стороні авторство довести не можна. Підпис створюють закритим ключем, а перевіряють відкритим: перевірити може будь-хто, а створити – тільки власник закритого ключа, що дає неспростовність і не потребує попереднього обміну секретом. Ціна – розмір і швидкість: тег має 16 байтів, підпис – 256 байтів.

ВИСНОВКИ

1. Складники навчального RSA перевірено на відомих значеннях: $7^{13} \bmod 100 = 7$, modinv(17, 3120) = 2753, контрольний приклад дає c = 2790 і повертає m = 65; 6 еталонних тестів каркаса проходять. Для розрядності варіанта (32 біти на множник) згенеровано ключі з модулем на 64 біти, для яких $e \cdot d \bmod \varphi(n) = 1$.

2. Навчальний RSA не є стійким: модуль варіанта розкладено ρ-методом за 26,3 мс, а перебором дільників – за 2,7 хв. Час перебору зростає приблизно в 16 разів на кожні 4 біти множника, тоді як час генерації простого числа – лише з 0,129 мс (32 біти) до 414 мс (1024 біти). Практичний захист потребує модуля від 2048 бітів і доповнення OAEP або PSS.
3. Швидке піднесення до степеня виконало 19 множень для e = 65537 і 102 для 64-бітового d проти 65 536 і близько $1{,}0 \cdot 10^{19}$ за наївного методу.
4. Реалізовано AEAD ChaCha20-Poly1305 з випадковим 96-бітовим nonce на кожне повідомлення: у 10 000 шифрувань отримано 10 000 різних nonce, усі 336 однобітових змін пакета та зміну асоційованих даних відхилено. Шифрування без тегу ту саму зміну прийняло без помилки.
5. Статичний IV у CBC розкриває повтори й спільні початки повідомлень (3 і 2 спільні блоки з 3, 3 різні шифротексти на 5 повідомлень); випадковий IV усуває обидва наслідки (0 спільних блоків, 5 різних шифротекстів).
6. MAC-then-Encrypt змушує приймача розшифрувати кожен із 96 змінених пакетів і дає дві розрізнювані відмови (24 і 72); Encrypt-then-MAC відхилив усі 96 пакетів однією відмовою без жодного розшифрування.
7. Підпис RSA-PSS (2048 бітів, SHA-256) має 256 байтів, перевіряється для коректного повідомлення й відхиляє всі 2048 однобітових змін підпису та всі 136 змін повідомлення. Написано 32 власні тести, усі проходять.

ПЕРЕЛІК ПОСИЛАНЬ

1. Ferguson N. Cryptography Engineering: Design Principles and Practical Applications / N. Ferguson, B. Schneier, T. Kohno. – Indianapolis : Wiley, 2010. – 384 p.
2. Katz J. Introduction to Modern Cryptography / J. Katz, Y. Lindell. – 3rd ed. – Boca Raton : CRC Press, 2020. – 648 p.
3. Rivest R. L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems / R. L. Rivest, A. Shamir, L. Adleman // Communications of the ACM. – 1978. – Vol. 21, № 2. – P. 120–126.
4. RFC 8439. ChaCha20 and Poly1305 for IETF Protocols / Y. Nir, A. Langley ; IRTF. – 2018.
5. RFC 8017. PKCS #1: RSA Cryptography Specifications Version 2.2 / K. Moriarty [et al.] ; IETF. – 2016.

6. NIST SP 800-57 Part 1 Rev. 5. Recommendation for Key Management: Part 1 – General / E. Barker. – Gaithersburg : NIST, 2020.
7. Bellare M. Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm / M. Bellare, C. Namprempre // Journal of Cryptology. – 2008. – Vol. 21, № 4. – P. 469–491.
8. NIST SP 800-38A. Recommendation for Block Cipher Modes of Operation: Methods and Techniques / M. Dworkin. – Gaithersburg : NIST, 2001.
9. Boudot F. Comparing the Difficulty of Factorization and Discrete Logarithm: A 240-Digit Experiment / F. Boudot [et al.] // Advances in Cryptology – CRYPTO 2020. – Cham : Springer, 2020. – P. 62–91.
10. Vaudenay S. Security Flaws Induced by CBC Padding – Applications to SSL, IPSEC, WTLS… / S. Vaudenay // Advances in Cryptology – EUROCRYPT 2002. – Berlin : Springer, 2002. – P. 534–545.
11. Соколовський В. В. Навчально-методичний комплекс дисципліни «Безпека програмного забезпечення»: код лабораторних робіт [Електронний ресурс]. – Режим доступу: https://github.com/sokolusha/software-security-course

ДОДАТОК А. ПРОГРАМНИЙ КОД

Повний програмний код роботи розміщено в репозиторії: https://github.com/kqjqxb/Software-Security, коміт 123d71c. Файли роботи – у каталозі `lab4/lab04_crypto`: `crypto_lab/aead.py`, `crypto_lab/aead_demo.py`, `crypto_lab/rsa_demo.py`, `crypto_lab/rsa_bench.py`, `crypto_lab/misuse_static_iv.py`, `crypto_lab/misuse_mac_order.py`, `crypto_lab/sign_demo.py`, `tests/test_variant.py`.